

WMATD UNIT - 2

LINK:-https://docs.google.com/document/d/1p_kK59k5GcQ4WZecGGiORK_4FnX8A-NSoiP_XsVWCiY/edit?usp=sharing

Foundational Concepts of Performance Testing

This section establishes the fundamental principles of performance testing, defining its scope, objectives, and the critical factors that influence an application's behavior under load. It frames performance testing not merely as a technical verification step but as a crucial discipline for ensuring the quality of service and user experience for modern web and mobile applications.

Defining Performance Testing: Core Concepts and Rationale

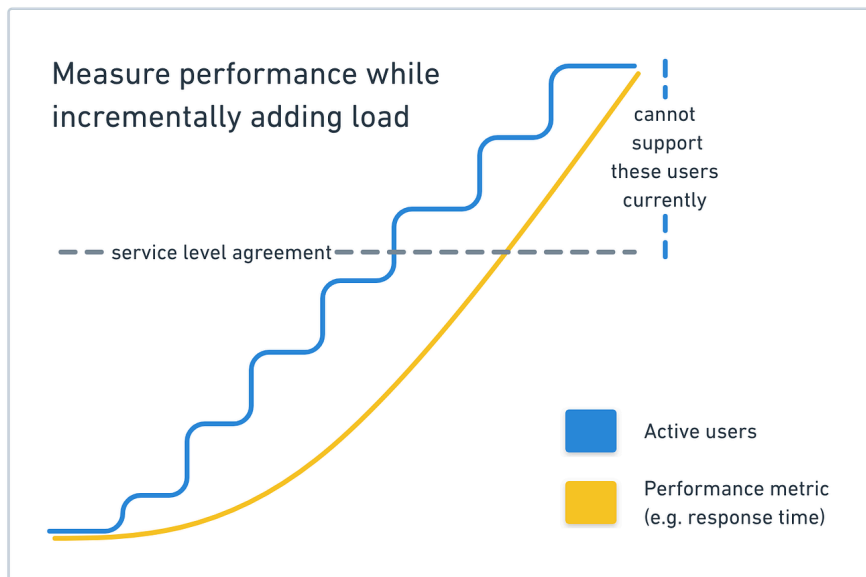
Performance testing is a non-functional testing discipline designed to determine the responsiveness, throughput, reliability, and scalability of a system under a given workload.¹ Its primary purpose is to ensure that a web or mobile application can effectively handle its expected load, particularly during peak usage hours, while continuing to serve users in a timely and reliable manner.¹ Unlike functional testing, which validates *what* an application does (e.g., "Does the login button work?"), performance testing evaluates *how well* the application does under specific conditions (e.g., "How long does it take to log in when 1,000 users are accessing the system simultaneously?").

In the context of contemporary web and mobile applications, performance is not a secondary technical attribute but a primary component of the user experience, making it a core feature of the product itself. The modern user has a low tolerance for delays; a slow or unresponsive application is often perceived as a broken one. This can lead directly to user abandonment, loss of revenue, and damage to brand reputation. The statement that a potential user "will move on to the competitor's site" if they have to wait too long for a service underscores the direct link between system performance and business success.¹ Consequently, performance testing is not an optional quality check but a critical validation of a core business requirement.

Primary Objectives of Performance Testing

The overarching goal of performance testing is to evaluate an application's performance with respect to realistic, real-world scenarios.¹ This goal is broken down into several specific objectives:

- **Evaluating System Performance Under Peak Load:** To measure and validate key performance indicators such as response time, system reliability, and resource availability during the busiest operational periods.
- **Identifying System Breaking Points:** To determine the specific load levels at which system performance begins to degrade unacceptably or at which the system fails entirely. This helps in understanding the system's operational limits.
- **Assessing Business Impact:** To analyze how degraded performance could negatively affect customer loyalty, sales conversions, and overall profitability, thereby providing a business case for optimization efforts.



[FIG] A conceptual diagram can be used here to illustrate the relationship between increasing user load and its impact on response time. The graph would show user load on the x-axis and response time on the y-axis. As the load increases, the response time should show a gradual, linear increase, followed by a sharp upward curve (the "knee"), indicating performance degradation, and finally a plateau or drop-off representing system failure or timeout errors.

Key Factors Influencing Application Performance

The performance of an application is not a single attribute but the result of a complex interplay of various factors. Performance tests are designed to manipulate and measure these variables. The most critical factors include ¹:

- **Response Time:** The total time elapsed from the moment a user sends a request until the system provides a complete response. This is the most direct measure of user-perceived speed.
- **Memory and CPU Utilization:** The amount of server-side computational resources (RAM and processor cycles) consumed by the application. High or escalating

consumption can indicate inefficient code, memory leaks, or inadequate hardware.

- **Network Bandwidth:** The data transfer capacity of the network infrastructure connecting the user to the application servers. Insufficient bandwidth can create a significant bottleneck, especially for data-intensive applications.
- **Number of Concurrent Users:** The quantity of users simultaneously interacting with the system. This is the most common variable manipulated in load and stress tests to simulate different traffic levels.
- **User Type and Behavior:** Different users perform different actions (e.g., browsing vs. purchasing), each placing a unique load on the system.
- **Download Time:** The time required to transfer all application assets (HTML, CSS, images, scripts) to the client.
- **Client-Side Configuration:** The hardware and software characteristics of the user's device (e.g., PC, mobile phone), which can affect rendering time and perceived performance.

These factors do not operate in isolation; they form an interdependent system where a change in one factor can cause a cascade of effects across the others. For example, an increase in the *number of concurrent users* will inevitably lead to higher *memory and CPU utilization* on the server. This increased resource consumption can cause processing queues to lengthen, which in turn increases the average *response time*. Concurrently, a higher user load will consume more *network bandwidth*. Therefore, a performance test is a controlled experiment designed to observe how this entire system of factors reacts to a specific stimulus, such as increased load. The goal of subsequent analysis is to diagnose which factor is the primary bottleneck—the weakest link constraining the system's overall performance.

Core Types of Performance Testing

Within the discipline of performance testing, two primary methodologies are employed to assess different aspects of an application's behavior: Load Testing and Stress Testing. This section provides a detailed examination of their distinct objectives and processes.

Load Testing: Verifying Performance Under Expected Load

Load testing is the process of evaluating an application's behavior under its anticipated real-world workload. It involves simulating a large number of users accessing the application simultaneously to test its performance at its *maximum expected load*.¹ The fundamental objective of load testing is not to break the system, but to verify that it can perform adequately and meet its specified performance requirements under its designed peak workload. It answers the question: "Can our system handle the traffic we expect on our busiest day?"

A successful load testing effort requires a systematic and structured process:

1. **Defining the Environment:** The first step is to establish a dedicated testing environment that meticulously simulates the production environment. This includes matching the hardware specifications (CPU, RAM), software versions (OS, database, web server), and network configuration of the live system. Failure to create an accurate replica makes it difficult to extrapolate test results to predict real-world performance.¹
2. **Defining the Strategy and User Load:** This phase involves designing test cases that model realistic user behavior. The number of concurrent users is determined by analyzing business requirements or historical data to approximate usage patterns for different parts of the application. The strategy also outlines which specific user actions (e.g., login, search, checkout) will be simulated to identify potential bottlenecks.¹
3. **Executing the Load Test:** Using specialized performance testing tools, the defined load is generated against the test environment. The test is typically run for a predetermined duration, and performance metrics are collected continuously. The process may be repeated multiple times to ensure the consistency and reliability of the results.¹

Stress Testing: Evaluating System Behavior Beyond Normal Limits

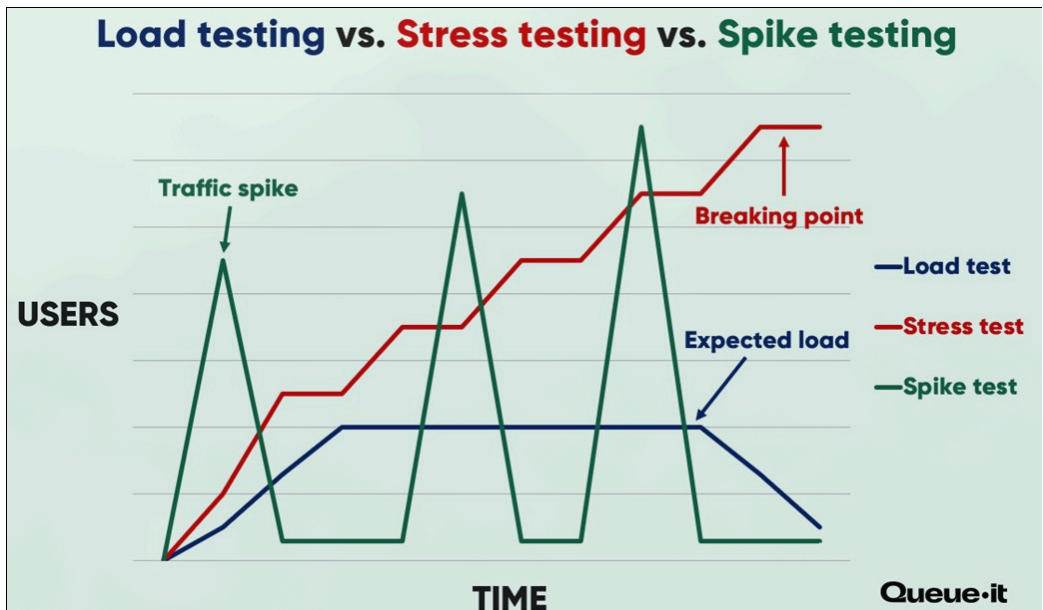
Stress testing is the process of subjecting an application to workloads that are *well beyond its specified operational limits*.¹ This can involve simulating an extreme number of concurrent users, flooding the system with a high volume of data, or running tests for extended periods to uncover issues like memory leaks. Unlike load testing, which validates expected behavior, stress testing is intentionally designed to push the system to its breaking point.

The primary objectives of stress testing are:

- **To Identify Failure Modes:** To determine precisely how and when the system fails when overloaded.
- **To Evaluate System Robustness:** To observe whether the system degrades gracefully (e.g., slows down but remains operational) or fails catastrophically (e.g., crashes, corrupts data).
- **To Verify Recovery Capabilities:** To monitor how effectively the system recovers and returns to a stable state after the extreme load is removed.

[FIG] A diagram can illustrate a typical load profile for a stress test. The graph would show user load on the y-axis and time on the x-axis. The load would ramp up to the expected peak (the load test level), hold there for a period, and then continue to ramp up significantly into the stress test zone. It would remain at this extreme level until performance degrades or the system fails, after which the load would be removed to observe the system's recovery time. The fundamental distinction between load and stress testing lies in the type of risk they are designed to mitigate. Load testing is concerned with mitigating known *business risk*. It validates that the system can meet its service level agreements (SLAs) and handle predictable peak traffic, such as a retail website during a holiday sale. In contrast, stress testing is

designed to mitigate unknown *technical risk*. It prepares the system for unpredictable events, such as a sudden traffic spike from a viral marketing campaign or a denial-of-service attack, ensuring the system remains robust and can recover from catastrophic failure.



Comparative Analysis: Load Testing vs. Stress Testing

To clarify the differences between these two critical testing types, the following table provides a side-by-side comparison of their key attributes.

Attribute	Load Testing	Stress Testing
Primary Objective	To verify system performance against predefined requirements under expected peak load.	To identify the system's failure point, observe its behavior under extreme conditions, and verify its recovery capabilities.
Load Level	At or just below the specified maximum expected load.	Well beyond the specified maximum load.
Duration	Typically for a defined period that simulates a peak business cycle (e.g., one hour).	Often for extended periods or until the system fails.
Expected Outcome	The system is expected to perform within acceptable limits for all key metrics (e.g., response time, error rate).	The system is expected to fail. The focus is on <i>how</i> it fails (gracefully or catastrophically) and how it recovers.
Risk Mitigation	Mitigates business risk by ensuring the system can handle its anticipated user base.	Mitigates technical risk by ensuring system robustness and resilience against unexpected events.

Performance Measurement, Metrics, and Analysis

Effective performance testing is a quantitative discipline that relies on the precise measurement and careful analysis of key metrics. This section details the specific data points collected during testing and explains how they are interpreted to generate actionable insights.

Identifying and Utilizing Potential Metrics

The selection of appropriate metrics is essential for quantifying the results of load tests and comparing system performance against established benchmarks.¹ These metrics provide the empirical data needed to assess whether an application meets its performance goals. The following table, adapted from the source material, categorizes the most critical metrics collected during performance testing.

Table 11.10. Load testing metrics		
S. No.	Metric	Description
Scalability and Usage		
1.	Web page views per week	It counts the number of web pages viewed per week.
2.	Web page views per hour	It counts the number of web pages viewed per hour.
3.	Web page views per second	It counts the number of web pages viewed per second.
4.	Web page hits per week	It counts the number of web pages hit per week.
5.	Web page hits per hour	It counts the number of web pages hit per hour.
6.	Web page hits per second	It counts the number of web pages hit per second.
7.	Average hits per web page	It is the ratio of the number of web page hits to the number of web page views
8.	Cycles	It counts the number of times the test is executed.

(Contd.)

S. No.	Metric	Description
Performance		
9.	Number of concurrent users	It counts the number of simultaneous users accessing a web page within a specified time interval.
10.	Response time	It measures the total time it takes after sending the request till the first response is received.
11.	Wait time	It measures the time between which the request was sent till the first byte is received.
12.	Throughput	It provides the amount of data sent by the application to the virtual users, measured in bytes.
13.	Elapsed time	It measures the time elapsed to complete a transaction, measured in seconds.
14.	Minimum web page run time	It measures the minimum time taken by a web page to execute in seconds.
15.	Maximum web page run time	It measures the maximum time taken by a web page to execute in seconds.
Failure		
16.	Failed hits per second	It counts the number of requests that fail per second.
17.	Connections failed	It counts the total number of connections that could not be established during the test.
18.	Failed cycles per second	It tests the number of rounds that failed per second.
19.	Maximum wait time before failure	It specifies the maximum time the user will wait before abandoning the web site.
Working with the load testing team earlier in the software life cycle is important, as it will help in identifying metrics and will ensure performance of the web site.		

Interpreting Test Results for Actionable Insights

The raw data collected during a performance test must be analyzed to determine if the system's performance meets user and business expectations. If the measured performance falls short of the specified targets, corrective action is required.¹

However, individual metrics often represent symptoms rather than root causes. The true skill in performance analysis lies in correlating different metrics to diagnose the underlying bottleneck. A single data point, such as a high response time, is not actionable on its own. It is a symptom that could be caused by an overwhelmed application server, a slow database query, network latency, or other factors. To pinpoint the root cause, an analyst must examine multiple metrics in conjunction. For example:

- **Scenario A: Application Bottleneck:** High response time correlated with high server CPU utilization but low database CPU usage points to inefficient application code or an underpowered application server.
- **Scenario B: Database Bottleneck:** High response time correlated with low server CPU

utilization but high database CPU usage suggests a database-level issue, such as an unoptimized query or insufficient indexing.

- **Scenario C: Network Bottleneck:** High response time correlated with low CPU usage on both the application and database servers may indicate network latency or insufficient bandwidth.

This diagnostic approach transforms performance testing from a simple pass/fail check into a sophisticated engineering process aimed at identifying and resolving the specific constraints that limit system performance.

The Role of Performance Testing Checklists

Table 11.11. Performance testing checklist

S. No.	Description	Yes/No/NA	Remarks
Users			
1.	Have the maximum number of users been identified for the web application?		
2.	Is complexity of the system determined?		
3.	Are peak hours identified?		
4.	Is the duration of session of users analyzed?		
5.	Have the type of users been identified?		
Response Time			
6.	Is the response time calculated?		

(Contd.)

S. No.	Description	Yes/No/NA	Remarks
7.	Is the maximum response time determined?		
8.	Does the response time meet the established threshold?		
9.	Are break points identified for planning load tests?		
10.	Are the causes of crash at the break points identified?		
Database			
11.	Is the maximum database capacity identified?		
12.	Are the past experiences with the database documented?		
13.	Is load sharing and balancing facility available?		
14.	Does the application read the database content correctly?		
15.	Does the application write the database content correctly?		

Tools	
16.	Are load testing tools identified?
17.	Are the employees trained?
18.	Is the load testing tool compatible with the platform?
19.	Is external support available for the tool?
General	
20.	Are there any disk space limitations?
21.	Are people with required skill sets available?
22.	Has the time and number of iterations required for testing been identified?
23.	Are rules for concurrency control being followed?
24.	Is the maximum wait time before failure determined?
25.	Are memory requirements and disk space usage identified?
Table 11.11 presents a generic list of performance testing questions that are common to most of the web applications. The performance testing checklist may help to uncover the major performance related problems early in software development process.	

To ensure a comprehensive and systematic approach, performance testing activities can be guided by a checklist. A checklist serves as a quality control mechanism for the testing process itself, ensuring that no critical area is overlooked during planning and execution. Key areas to address in a performance testing checklist include ¹:

- **Users:** Has the expected number of concurrent users been determined? Has the duration of a typical user session been analyzed?
- **Response Time:** Has the maximum acceptable response time been defined and agreed upon? Does the measured response time meet this established threshold?
- **Database:** Is a load sharing and balancing facility available? Does the application read from and write to the database correctly under load?
- **Tools:** Is the selected load testing tool compatible with the application's technology stack and platform? Are team members adequately trained to use the tool?

DATABASE TESTING

Testing data-centric web applications is important to ensure their error-free operation and increased customer satisfaction. For example, consider the example for purchasing items from an online store. If the user performs a search based on some keywords and price preferences, a database query is created by the database server. Suppose due to some programming fault in the query, the query does not consider the price preferences given by the customer, this will produce erroneous results. These kinds of faults must be tested and removed during database testing.

Testing data-centric web applications is important to ensure their error-free operation and increased customer satisfaction. For example, consider the example for purchasing items from an online store. If the user performs a search based on some keywords and price preferences, a database query is created by the database server. Suppose due to some programming fault in the query, the query does not consider the price preferences given by the customer, this will produce erroneous results. These kinds of faults must be tested and removed during database testing.

Important issues in database testing may include:

- (i) Data validation
- (ii) Data consistency
- (iii) Data integrity
- (iv) Concurrency control and recovery
- (v) Data manipulation operations such as addition, deletion, updation and retrieval of data.
- (vi) Database security

A database must be tested for administrative level operations such as adding, deleting and updating an item in the database, and user operations such as searching an item from the database or providing personal details. In the example of the online shopping website, the most common administrative operations and user operations include:

Administrative operations

- (i) Inserting a new item into the database
- (ii) Deleting an existing item from the database
- (iii) Updating an existing item from the database
- (iv) Viewing an existing item from the database

User operations

- (i) Searching items from the database
- (ii) Registering into the website involves storing the user's personal details
- (iii) Placing an order involves storing user preferences and purchase details into the database

Tooling for Performance Measurement and Profiling

Modern performance testing is heavily reliant on specialized software tools. It is practically impossible to manually simulate the load of hundreds or thousands of concurrent users required to accurately assess the performance of contemporary web and mobile applications.¹ This section provides an overview of these essential tools.

Overview of Dynamic Performance Testing Tools

Dynamic software testing tools are used to execute the application and evaluate its performance under simulated load and stress conditions.¹ These tools are the cornerstone of performance testing, enabling testers to create controlled, repeatable, and scalable experiments. Their primary functions include:

- **Load Generation:** Simulating network traffic from a large number of "virtual users," each executing a predefined set of actions.
- **Transaction Scripting:** Allowing testers to define and script the specific business processes that virtual users will follow (e.g., searching for a product, adding an item to a shopping cart, completing a purchase).
- **Metric Collection:** Automatically capturing and aggregating the key performance and failure metrics discussed in the previous section.
- **Reporting and Analysis:** Presenting the collected data through comprehensive reports, graphs, and tables to facilitate analysis and bottleneck identification.

The fundamental value of these tools lies in their ability to create scientific and repeatable experiments. Manual testing by a large group of people is expensive, difficult to coordinate, and yields subjective results that are not easily measured. In contrast, a performance tool can simulate thousands of users from a single machine, ensuring that each virtual user executes the exact same steps in every test run. This repeatability is crucial for A/B testing optimizations: a test can be run, a system change can be made, and the test can be run again to precisely measure the impact of the change. This transforms performance testing from an ad-hoc activity into a rigorous engineering discipline.

Examples of Commercial and Open-Source Tools

A variety of powerful performance testing tools are available, both commercially and as open-source projects. The source material lists several popular examples that represent the state of the industry¹:

- **Micro Focus LoadRunner** (formerly Mercury Interactive's Load Runner): A long-standing and feature-rich commercial tool.
- **Apache JMeter:** A widely used, powerful open-source tool from the Apache Software Foundation.
- **Segue Software's Silk Performer:** A commercial tool focused on enterprise application testing.
- **IBM Rational Performance Tester:** A commercial tool that is part of IBM's broader suite of software development and testing products.
- **Compuware's QALOAD:** A commercial tool for load and performance testing.

Performance Optimization Techniques

The ultimate goal of performance testing is not merely to find problems but to drive their resolution. This section outlines how test results are translated into concrete performance improvements through bottleneck analysis, system tuning, and risk management.

Using Test Results to Identify Performance Bottlenecks

The diagnostic process of correlating performance metrics is the primary method for identifying bottlenecks. By analyzing the data from load and stress tests, engineers can pinpoint the specific component—be it the application code, database, network infrastructure, or a third-party service—that is constraining the overall performance of the system.

Corrective Actions and System Tuning

Once a bottleneck has been identified, a range of corrective actions can be implemented to address the issue. The appropriate action depends on the nature of the bottleneck:

- **Code Optimization:** This involves rewriting inefficient algorithms, optimizing slow database queries (e.g., by adding indexes), or fixing memory leaks in the application code.
- **Resource Scaling:** If the bottleneck is due to insufficient hardware capacity, resources can be scaled up. This can be done through *vertical scaling* (adding more CPU or memory to an existing server) or *horizontal scaling* (adding more servers to a cluster).
- **System Configuration Tuning:** Performance can often be improved by adjusting the configuration parameters of the underlying software stack. This includes tuning database connection pools, web server thread counts, application caching settings, and operating system parameters.

Risk Analysis for Acceptable Performance Degradation

It is not always feasible or cost-effective to engineer a system that shows no performance degradation under any conceivable load. After conducting stress tests, the testing team must analyze the points at which performance degrades and compare them with what is considered acceptable for the system.¹ Risk analysis is used to inform this business decision. For instance, a business may decide that a two-second increase in response time during an extremely rare traffic spike is an acceptable risk when weighed against the significant cost of engineering the system to handle such an anomaly without any degradation. This is a

strategic decision informed by the technical data provided by stress testing.

Special Considerations for Mobile Performance Testing

While the fundamental principles of performance testing are universal, applying them to mobile applications introduces a unique set of challenges and considerations. The core concepts of load and stress testing remain critical, as the backend servers powering mobile apps must be tested for concurrent user load just like web servers. However, the client is no longer a powerful desktop machine on a stable network but a resource-constrained mobile device on a variable and often unreliable network.

Unique Challenges in the Mobile Environment

The client-side environment for mobile applications is fundamentally uncontrolled and can be hostile to performance. Web performance testing often assumes a reasonably powerful client machine with a stable, high-speed internet connection. Mobile testing cannot make these assumptions. Mobile devices have limited CPU power, memory, and battery life, and their network connectivity can fluctuate dramatically between high-speed Wi-Fi and slow, high-latency cellular data.¹

This reality requires that mobile performance testing be divided into two distinct domains:

1. **Server-Side Performance:** This involves traditional load and stress testing of the backend APIs that the mobile application communicates with. The objectives are the same as for web applications: to measure response time, throughput, and scalability under load.
2. **Client-Side Performance:** This focuses on how the application itself performs on the physical device. This introduces a new set of critical metrics that are not typically central to web testing:
 - **Battery Consumption:** An application that functions quickly but rapidly drains the user's battery is considered a performance failure.
 - **On-Device CPU and Memory Usage:** An application that consumes excessive resources, making the entire device feel sluggish, will likely be uninstalled by the user.
 - **Rendering Time:** This measures how quickly the application's user interface (screens, lists, animations) draws on the device's screen.
 - **Network Robustness:** This evaluates how the application behaves when the network connection is slow, intermittent, or drops completely. A well-behaved application should handle these conditions gracefully (e.g., by displaying a message and allowing retries) rather than crashing or becoming unresponsive.